

Carpooling Application — Complete System Documentation

Project Name: ReachNative Car (Carpooling App) **Version:** 1.0.0 **Date:** June 2026 **Production URL:** <https://carpooling-app-3-virid.vercel.app>
Repository: <https://github.com/youssef67e7/carpooling-app-3.git>

Table of Contents

1. Project Summary
2. Architecture
3. System Design
4. Technical Structure
5. Implementation & UI
6. API Reference
7. Features Status
8. Testing
9. Deployment
10. Production Polish
11. Changelog

12. Conclusion & Future Work

1. Project Summary

Overview

Full-stack ride-hailing application. Passengers request rides, drivers accept and complete trips. No real-time sockets — polling + FCM push notifications.

Tech Stack

Layer	Technology
Backend runtime	Node.js (vanilla, no framework)
HTTP server	Express.js
Database	MongoDB Atlas (cloud) + in-memory fallback
ODM	Custom in-memory ODM over native MongoDB driver
Frontend (mobile)	Flutter with Riverpod state management
Frontend (admin)	Vanilla HTML/JS
Maps	OpenStreetMap via <code>flutter_map</code>
Push notifications	Firebase Cloud Messaging v1 HTTP API
Email	Nodemailer + Gmail SMTP
SMS	Firebase phone auth (blocked — billing not enabled)
Auth	JWT (15 min) + refresh tokens (7 days, rotated)
Deployment	Vercel (serverless functions)

Feature Completion: 87% (61/70)

Phase	Complete	Total
Phase 6 — Auth	6	6
Phase 7 — Notifications	12	14
Phase 8 — Ride Experience	9	10
Phase 9 — Maps	7	7
Phase 10 — Wallet	9	9
Phase 11 — Driver Features	8	8
Phase 12 — Admin	7	9
Phase 13 — Production	3	7

Missing Items

1. Driver rates passenger (endpoint + UI)
 2. Admin broadcast notifications (POST /admin/broadcast)
 3. Refund processed FCM notification
 4. Report resolved FCM notification
 5. CSV export endpoints
 6. Crash reporting, analytics, monitoring
-

2. Architecture

Stack

Layer	Technology
Backend runtime	Node.js (vanilla, no framework)
HTTP server	Express.js
Database	MongoDB Atlas (cloud) + in-memory fallback
ODM	Custom in-memory ODM (backend/src/mongo/odm.js) over native MongoDB driver
Frontend (mobile)	Flutter with Riverpod state management
Frontend (admin)	Vanilla HTML/JS (backend/public/admin-ui/)
Maps	OpenStreetMap via flutter_map
Push notifications	Firebase Cloud Messaging (FCM v1 HTTP API)
Email	Nodemailer + Gmail SMTP
SMS	Firebase phone auth (blocked — billing not enabled)
Auth	JWT (15 min) + refresh tokens (7 days, rotated)
Deployment	Vercel (serverless functions)

Folder Structure

```

backend/
  src/
    routes/      # Express route handlers (auth, rides, admin, wallet, etc.)
    models/      # ODM model definitions (User, Ride, WalletAccount, etc.)
    mongo/
      odm.js      # Custom ODM – Mongoose-like API over native driver
      queries/    # Low-level native driver queries (users.js, rides.js, etc.)
      schema.js   # ensureMongoIndexes() – all index creation
      client.js   # MongoDB client connection
    middleware/   # authRequired, blockCheck, roleRequired, rateLimiters
    services/     # Business logic: fcmService, walletLedger, driverRating, sendEmail, etc
    utils/        # signUserToken, logger, googleSignIn helpers, etc.
    errors/       # AppError class
    config/       # fixedAdmins, env config
    schemas/      # express-validator schemas
    scripts/      # init-indexes.js, seed.js
    public/
      admin-ui/   # Vanilla web admin panel

apps/mobile-flutter/
  lib/
    core/
      api/        # ApiClient (Dio), auth_interceptor, api_endpoints
      providers/  # Riverpod providers (auth, ride, wallet, driver, admin)
      services/   # FCM, token manager, session reset, upload, location tracking
      theme/      # WeretTokens design tokens
      utils/      # Logging, navigation, geo helpers
      constants/  # Vehicle types, etc.
    features/
      auth/       # Login, register, settings, screens
      driver/     # Driver-specific screens
      wallet/     # Wallet screens (both roles)
      more/       # Passenger menus, earnings, ratings, cars
      debug/      # Debug log viewer
    shared/
      models/     # WeretUser model
      widgets/    # Reusable widgets (map, buttons, cards, OTP input, etc.)
    l10n/         # English + Arabic translations
    core/router/  # GoRouter configuration (app_router.dart)

```

Data Flow

Authentication

1. User submits credentials → `POST /auth/*` → backend validates → creates JWT + refresh token
2. Flutter stores tokens via `TokenManager` (SharedPreferences)

3. Every API call: `AuthInterceptor` attaches `Bearer` token; on 401, auto-refreshes via `POST /auth/refresh`
4. On app start: `hydrate()` calls `GET /auth/me` with stored token → restores session

Ride Flow (no real-time sockets)

1. Passenger creates ride → `POST /rides/create`
2. Driver polls `GET /rides/available` (or screen refreshes every 15-30s)
3. Driver accepts → `POST /rides/:id/accept` → FCM sent to passenger
4. Driver arrives → `POST /rides/:id/arriving` → FCM to passenger
5. Passenger onboard → `POST /rides/:id/onboard` → FCM to passenger
6. Driver starts trip → `POST /rides/start` → FCM to passenger
7. Driver ends trip → `POST /rides/end` → FCM to both, payment ledger updated

Wallet

- Transactions recorded in `transactions` collection via `walletLedger.js`
- Passenger wallet accounts + driver earnings share same data model
- Withdrawals require OTP confirmation (2-step)

Push Notifications

- FCM tokens registered at login via `POST /auth/register-token`
- Backend sends via FCM v1 HTTP API in `fcmService.js`
- Each event has a dedicated `notify*` helper in `notificationHelpers.js`
- Flutter `FcmService` handles foreground (in-app banner), background (tap → ride-chat), and terminated (pending navigation)

Auth Flow

```
Login → generateAccessToken (15min) + generateRefreshToken (7d, 64-byte hex, SHA-256 hashed)
      → store in refreshTokens collection with TTL
      → return both to client
```

```
API Call → AuthInterceptor attaches accessToken
          → 401 → auto-call POST /auth/refresh
          → generate new accessToken + rotate refreshToken (delete old, store new)
          → retry original request
```

```
Logout → revokeAllRefreshTokens(userId)
        → clear local tokens
```

3. System Design

3.1 Use Case Diagrams

Actors

Actor	Description
Passenger	Requests rides, rates drivers, manages wallet, views history
Driver	Accepts rides, manages vehicles, toggles availability, views earnings
Admin	Manages users, rides, reports, transactions; approves drivers
System	Handles auth, notifications, payments, maps

Use Cases by Actor

Passenger

- Register / Login (Email OTP, Google, Phone)
- Request a ride
- Cancel a ride
- Rate driver
- View ride history
- Manage wallet (top up, withdraw)
- Send chat messages
- Report an issue
- Delete account

Driver

- Register / Login
- Submit driver application with documents
- Toggle online/offline status
- View available ride requests
- Accept ride

- Update ride status (arriving, onboard, start trip, end trip)
- Cancel ride
- Manage vehicles (add, set active)
- View earnings dashboard
- View ratings received
- Withdraw earnings
- Send chat messages

Admin

- Login (email/password or Google)
- View dashboard statistics
- Search and moderate users (block, verify, approve driver, delete)
- Search rides
- Manage reports (open, reviewing, resolved, dismissed)
- View and flag suspicious transactions
- View audit logs
- Delete users (cascade)

3.2 Database Schema (ERD)

Collections

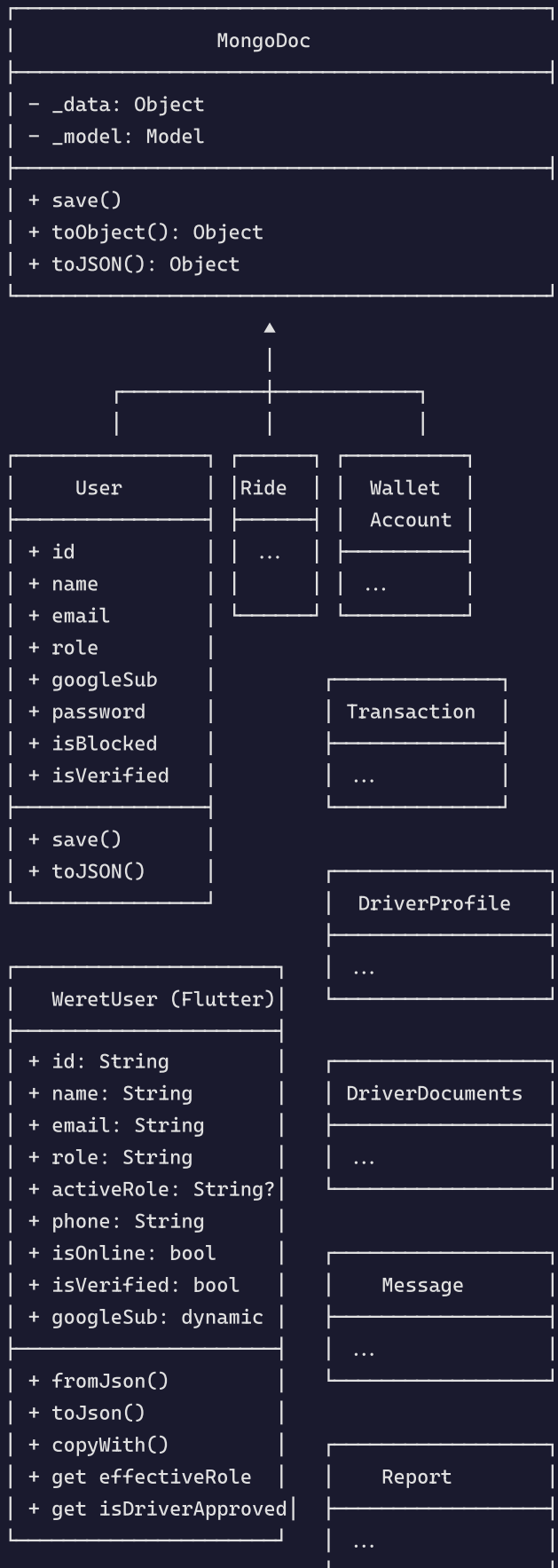
Collection	Purpose	Key Fields	Key Indexes
<code>users</code>	User accounts	name, email, password, role, google_sub, is_blocked	email (unique sparse), google_sub (unique sparse), role
<code>rides</code>	Ride records	passenger_id, driver_id, status, pickup, destination, fare	passenger_id+status, driver_id+status, 2dsphere
<code>wallet_accounts</code>	Wallet funding sources	user_id, type, label, details	user_id
<code>transactions</code>	All financial transactions	user_id, type, amount, direction, flagged	user_id+created_at
<code>driverProfiles</code>	Driver-specific data	userId, isOnline, isAvailable, currentLocation	userId (unique), isOnline+isAvailable, 2dsphere
<code>driverDocuments</code>	Driver document uploads	userId, licenseImage, vehicleRegImage, status	userId (unique)
<code>fcmTokens</code>	Push notification tokens	userId, token, platform	userId, token (unique)
<code>notifications</code>	Stored notifications	userId, type, title, body, read	userId+createdAt, userId+read
<code>refreshTokens</code>	JWT refresh tokens	userId, tokenHash, expiresAt	tokenHash (unique), expiresAt (TTL 7d)
<code>messages</code>	Ride chat messages	rideId, senderId, content, createdAt	rideId+createdAt
<code>bookings</code>	Pooling bookings	rideId, passengerId, status, seats	rideId
<code>adminAuditLogs</code>	Admin action audit trail	actorAdminId, action, targetType, targetId	createdAt (TTL 30d)
<code>reports</code>	User reports	reporterId, reportedUserId, rideId, reason, status	—
<code>withdrawalRequests</code>	Withdrawal requests	userId, amount, accountId, otpHash, status	—

Key Relationships

- **users** 1:N **rides** (as passenger via `passenger_id` , as driver via `driver_id`)
- **users** 1:N **wallet_accounts** (via `user_id`)
- **users** 1:N **transactions** (via `user_id`)
- **users** 1:1 **driverProfiles** (via `userId`)
- **users** 1:1 **driverDocuments** (via `userId`)
- **users** 1:N **fcmtokens** (via `userId`)
- **users** 1:N **notifications** (via `userId`)
- **rides** 1:N **messages** (via `rideId`)
- **rides** 1:N **bookings** (via `rideId`)

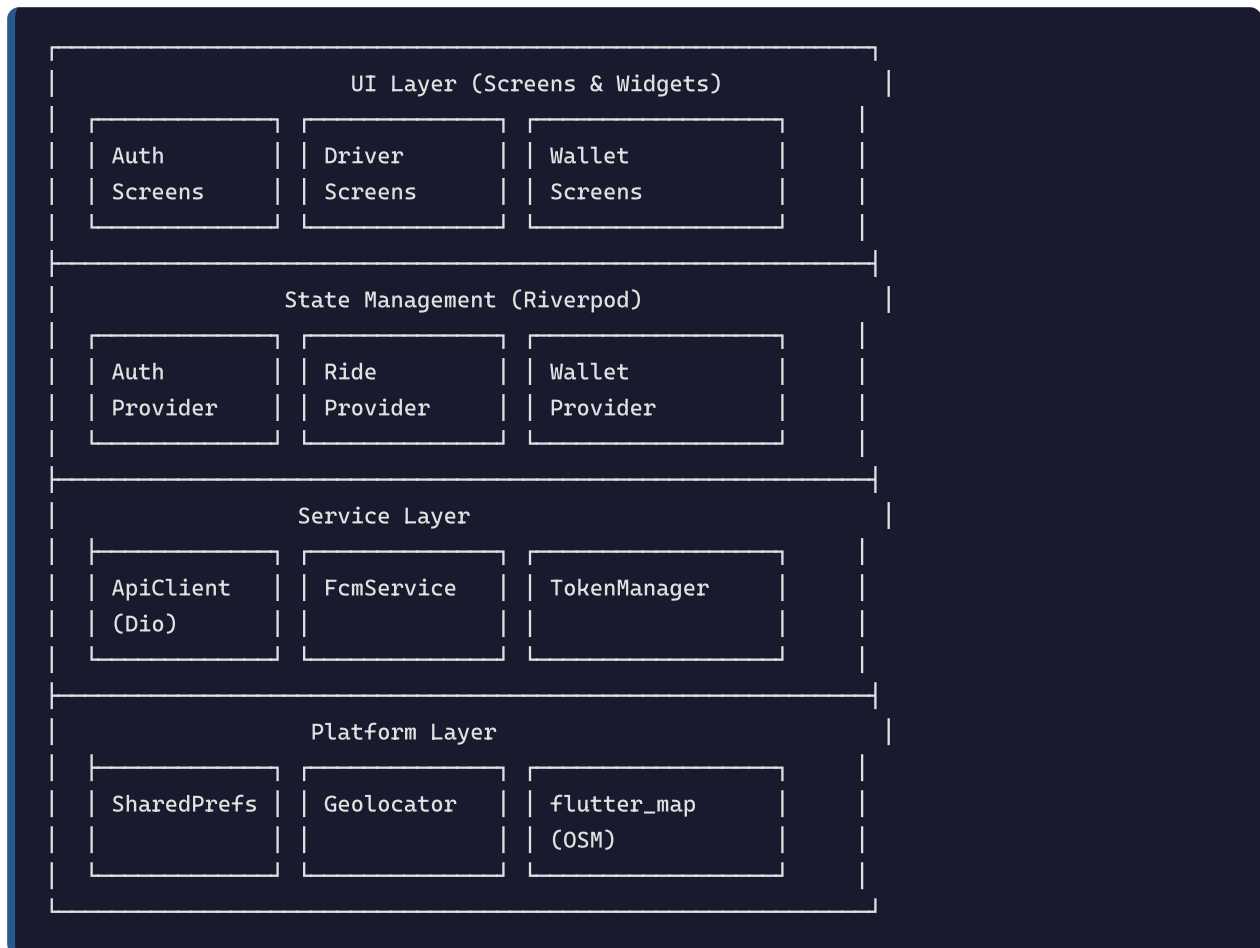
3.3 Class Diagrams

Core Data Models (ODM Layer)



4. Technical Structure

Flutter Architecture



Project Structure (Flutter)

```

apps/mobile-flutter/lib/
├─ core/
│   ├─ api/
│   │   ├─ api_client.dart          # Dio HTTP client wrapper
│   │   ├─ api_endpoints.dart       # All endpoint constants
│   │   └─ auth_interceptor.dart    # Token attach + refresh + block handling
│   ├─ providers/
│   │   ├─ auth_provider.dart       # Auth state + methods
│   │   ├─ ride_provider.dart       # Ride lifecycle state
│   │   ├─ wallet_provider.dart     # Wallet accounts + transactions
│   │   ├─ driver_provider.dart     # Driver dashboard + cars + earnings
│   │   └─ ui_provider.dart         # Theme mode
│   ├─ services/
│   │   ├─ fcm_service.dart         # Firebase Cloud Messaging
│   │   ├─ token_manager.dart       # JWT + refresh token storage
│   │   ├─ session_reset.dart       # Clear all providers on logout
│   │   ├─ upload_service.dart      # Image upload via backend proxy
│   │   └─ driver_location_tracker.dart # GPS streaming
│   ├─ router/
│   │   └─ app_router.dart          # GoRouter with 3 role-based shells
│   ├─ theme/
│   │   └─ weret_tokens.dart        # Design tokens (colors, spacing)
│   ├─ utils/
│   │   ├─ map_provider.dart        # OSM tile config
│   │   ├─ map_coords.dart         # Coordinate helpers
│   │   ├─ route_polyline.dart     # Route fetching/decoding
│   │   ├─ geo_helpers.dart        # Extract locations from rides
│   │   ├─ logout_action.dart      # Logout + navigation
│   │   ├─ api_error_message.dart  # Error message localization
│   │   ├─ show_alert.dart         # Alert dialog helper
│   │   └─ debug_logger.dart       # Singleton debug logger
│   └─ constants/
│       └─ vehicle_types.dart      # Vehicle type definitions
├─ features/
│   ├─ auth/                        # Login, register, settings, onboarding
│   ├─ driver/                      # Driver-specific screens
│   ├─ wallet/                      # Wallet screens (both roles)
│   ├─ more/                        # Menus, earnings, ratings, cars
│   └─ debug/                      # Debug log viewer
├─ shared/
│   ├─ models/
│   │   └─ weret_user.dart         # WeretUser model (Equatable)
│   └─ widgets/
│       └─ ...                     # 20+ reusable widgets
└─ l10n/
    ├─ en.json                    # English translations (930+ keys)
    └─ ar.json                    # Arabic translations (930+ keys)

```


Riverpod State Management

The app uses **Riverpod** (`flutter_riverpod`) with `StateNotifier` pattern for all state management.

Key Providers

Provider	Type	Purpose
<code>authProvider</code>	<code>StateNotifierProvider</code>	Auth state (user, token, session)
<code>rideProvider</code>	<code>StateNotifierProvider</code>	Ride lifecycle (active, available, history)
<code>walletProvider</code>	<code>StateNotifierProvider</code>	Wallet accounts and transactions
<code>driverProvider</code>	<code>StateNotifierProvider</code>	Driver dashboard, cars, earnings
<code>themeModeProvider</code>	<code>StateNotifierProvider</code>	Light/dark/system theme
<code>apiClientProvider</code>	<code>FutureProvider</code>	Singleton ApiClient (Dio)

Pattern

```
// Provider definition
final authProvider = StateNotifierProvider<AuthNotifier, AuthState>((ref) {
  return AuthNotifier(ref);
});

// State class
class AuthState {
  final WeretUser? user;
  final String? token;
  final bool hydrated;
  final bool loading;
  final String? error;
}

// Notifier class
class AuthNotifier extends StateNotifier<AuthState> {
  Future<void> loginEmail(String email, String password) async {
    state = state.copyWith(loading: true, clearError: true);
    try {
      final api = await _api;
      final data = await api.postJson(ApiEndpoints.authLogin, {...});
      await applySession(token: token, refreshToken: refreshToken, user: user);
    } catch (e) {
      state = state.copyWith(loading: false, error: localizedApiError(e));
      rethrow;
    }
  }
}
```

State Flow Example (Login)

```
User taps "Login"
  → authProvider.loginEmail(email, password)
  → state = AuthState(loading: true)
  → ApiClient.postJson('/auth/login', body)
  → Dio send request → interceptor attaches token
  → Response: { user, accessToken, refreshToken }
  → TokenManager.saveTokens(accessToken, refreshToken)
  → _cacheUser(user) to SharedPreferences
  → state = AuthState(user: user, token: accessToken, loading: false)
  → UI rebuilds via ref.watch(authProvider)
  → GoRouter redirects to /passenger/home or /driver/home
```

Dio Networking

ApiClient

- Singleton wrapped in a Riverpod `FutureProvider`
- Base URL: Configurable (production or localhost)
- Methods: `getJson` , `postJson` , `patchJson` , `delete` , `postMultipart`
- Automatic error handling and logging

AuthInterceptor

- **onRequest**: Attaches `Authorization: Bearer <accessToken>` to every request
- **onError (401)**: Auto-refreshes token by calling `POST /auth/refresh` , then retries original request. Avoids infinite refresh loops.
- **onError (403)**: Detects `ACCOUNT_BLOCKED` / `ACCOUNT_SUSPENDED` — clears local session, shows snackbar, navigates to login
- **onError (others)**: Passes through for provider-level handling



UI Architecture

Routing (GoRouter)

Three role-based shells with `StatefulShellRoute.indexedStack`:

- **Top-Level Routes** (pre-auth): `/onboarding` , `/login` , `/forgot-password` , `/register` , etc.
- **Passenger Shell** (4 tabs): `/passenger/home` , `/passenger/history` , `/passenger/more` , `/passenger/settings`
- **Driver Shell** (3 tabs): `/driver/home` , `/driver/earnings` , `/driver/profile`
- **Admin Shell** (5 tabs): `/admin/dashboard` , `/admin/users` , `/admin/rides` , `/admin/more` , `/admin/settings`

Widget Hierarchy

```
Scaffold
├── WeretAmbientBackground
│   └── SafeArea
│       └── WeretListScreen
│           └── Column
│               ├── WeretPageTitle
│               ├── WeretSectionCard
│               │   ├── CustomButton
│               │   └── WeretTextField
│               └── ...
```

Translation (easy_localization)

- Two locales: English (`en.json`) and Arabic (`ar.json`)
 - ~930 translation keys each
 - Arabic is RTL-compatible
 - Language switch via `SettingsScreen` toggle
-

5. Implementation & UI

Technology Stack

Backend

Component	Technology	Version
Runtime	Node.js	22.x
HTTP Framework	Express.js	4.x
Database	MongoDB Atlas (M0 Free)	7.x
ODM	Custom in-memory ODM	—
Auth	JWT (jsonwebtoken)	9.x
Password Hashing	bcryptjs	2.x
Push Notifications	Firebase Admin / FCM v1 HTTP	—
Email	Nodemailer + Gmail SMTP	6.x
File Upload	Cloudinary (base64)	—
Rate Limiting	express-rate-limit	8.x
Validation	express-validator	7.x
Deployment	Vercel (serverless)	—

Frontend (Flutter)

Component	Technology	Version
Framework	Flutter	3.x
State Management	Riverpod (flutter_riverpod)	2.x
Routing	go_router	14.x
HTTP Client	Dio	5.x
Maps	flutter_map (OpenStreetMap)	7.x
Push Notifications	firebase_messaging	15.x
Google Sign-In	google_sign_in	6.x
Local Storage	shared_preferences	2.x
GPS	geolocator	12.x
Localization	easy_localization	3.x
Cached Images	cached_network_image	3.x

Admin Panel

- Vanilla HTML, CSS, JavaScript
- Google Sign-In + email/password login
- Served as static files from `/admin-ui/`

State Management Scenarios

Scenario 1: User Login Flow

```
1. User enters email on LoginScreen
2. _sendEmailOtp() called
   → authProvider.requestEmailOtp(email)
   → POST /auth/email/send-otp
   → Backend generates 6-digit OTP, sends via email
   → Response: { success: true }
3. OTP input field appears
4. User enters OTP code
5. _verifyEmailOtp() called
   → authProvider.verifyEmailOtp(email, code)
   → POST /auth/email/verify-otp
   → Backend validates OTP, finds or creates user, issues JWT
   → Response: { data: { user, accessToken, refreshToken } }
6. applySession() saves tokens + user
   → TokenManager.saveTokens(accessToken, refreshToken)
   → _cacheUser(user) to SharedPreferences
   → state = AuthState(user: user, token: token, hydrated: true)
7. GoRouter redirects based on user.role
   → passenger → /passenger/home
   → driver → /driver/home
   → admin → /admin/dashboard
```

Scenario 2: Ride Request and Completion

1. Passenger selects pickup/destination, taps "Request Ride"
 - `rideProvider.createRide(body)`
 - `POST /rides/create`
 - Response: ride object (status: "pending")
2. Driver polls `GET /rides/available` every 15-30s
 - `rideProvider.fetchAvailableRides()`
 - List of available rides displayed
3. Driver taps "Accept"
 - `rideProvider.acceptRide(rideId)`
 - `POST /rides/:id/accept`
 - Backend updates ride status to "accepted"
 - Backend calls `notifyRideAccepted()` → FCM to passenger
 - Response: updated ride
4. Passenger receives FCM notification
 - `FcmService.onMessage` shows in-app banner
 - `rideProvider.fetchMyActiveRide()` refreshes state
5. Driver progresses through states:
 - `driverArriving()` → `POST /rides/:id/arriving` → FCM to passenger
 - `passengerOnboard()` → `POST /rides/:id/onboard` → FCM to passenger
 - `startRide()` → `POST /rides/start` → FCM to passenger
 - `endRide()` → `POST /rides/end` → FCM + payment ledger
6. After ride completes, passenger rates driver
 - `rideProvider.rateRide(rideId, rating, review)`
 - `POST /rides/rate`
 - Backend updates driver rating

Scenario 3: Token Refresh

1. User opens app → `hydrate()` called
 - `TokenManager.getAccessToken()` returns stored token
 - `GET /auth/me` with Bearer token
2. If token expired:
 - `AuthInterceptor` catches 401
 - `TokenManager.getRefreshToken()`
 - `POST /auth/refresh` with `refreshToken`
 - Backend rotates refresh token (SHA-256 hash, delete old, store new)
 - Response: { `accessToken`, `refreshToken` }
 - `TokenManager.saveTokens(new tokens)`
 - Dio retries original request (`GET /auth/me`)
3. User session restored
 - `state = AuthState(user: freshUser, token: newToken, hydrated: true)`

Scenario 4: Wallet Deposit

1. User taps "Top Up" on wallet screen
 - Navigate to WalletDepositScreen
2. User selects account, enters amount
 - `walletProvider.deposit(amount, accountId)`
 - `POST /wallet/deposit`
 - Backend creates transaction, updates balance
 - Backend calls `notifyWalletDeposit()` → FCM
 - Response: transaction object
3. Success modal displayed
 - `walletProvider.fetchTransactions()` refreshes history
 - `walletProvider.fetchAccounts()` refreshes balance

Scenario 5: Admin Blocks User

1. Admin searches for user in AdminUsersScreen
 - `GET /admin/users?search=user@email.com`
2. Admin taps "Block" on user card
 - `PATCH /admin/users/:id { is_blocked: true }`
 - Backend sets `is_blocked=true` on user
3. Blocked user's next API call:
 - `GET /auth/me` or any protected route
 - `blockCheck` middleware returns 403 "Account blocked"
4. Flutter AuthInterceptor catches 403 ACCOUNT_BLOCKED
 - `clearLocalSession()` removes all tokens + cached user
 - Snackbar: "This account is blocked. Contact support."
 - Navigate to `/login`

6. API Reference

Base URL: `https://carpooling-app-3-virid.vercel.app/api`

Auth: Most endpoints require `Authorization: Bearer <accessToken>` header. **Refresh:** `POST /auth/refresh` with `{ refreshToken }` in body. **Errors:** All errors return `{ error: "message" }` with appropriate HTTP status.

Auth Endpoints

Method	Endpoint	Auth	Description
GET	/health / /api/health	Public	DB status + collection counts
GET	/auth/google-config	Public	Google OAuth client IDs
POST	/auth/register	Public	Create account
POST	/auth/login	Public	Admin web login
POST	/auth/google	Public	Google sign-in
POST	/auth/phone/otp	Public	Request phone OTP
POST	/auth/phone/verify	Public	Verify phone OTP
POST	/auth/email/send-otp	Public	Request email OTP
POST	/auth/email/verify-otp	Public	Verify email OTP
GET	/auth/me	Required	Get current user
PATCH	/auth/profile	Required	Update profile
POST	/auth/verify-password	Required	Verify password
POST	/auth/delete-account	Required	Self-delete account
POST	/auth/forgot-password	Public	Request password reset OTP
POST	/auth/reset-password	Public	Reset password with OTP
POST	/auth/refresh	Public	Refresh token rotation
POST	/auth/logout	Required	Revoke all refresh tokens
POST	/auth/register-token	Required	Register FCM push token
POST	/auth/verify-firebase-phone	Public	Verify Firebase phone auth

Ride Endpoints

Method	Endpoint	Auth	Description
POST	<code>/rides/create</code>	Passenger	Create ride request
GET	<code>/rides/available</code>	Driver	Get available rides
GET	<code>/rides/my-active</code>	Required	Get current active ride
GET	<code>/rides/:id</code>	Required	Get ride details
GET	<code>/rides/:id/status</code>	Required	Get ride status
POST	<code>/rides/:id/accept</code>	Driver	Accept ride
POST	<code>/rides/:id/arriving</code>	Driver	Driver arrived
POST	<code>/rides/:id/onboard</code>	Driver	Passenger onboard
POST	<code>/rides/start</code>	Driver	Start trip
POST	<code>/rides/end</code>	Driver	End trip, process payment
POST	<code>/rides/:id/cancel</code>	Passenger	Cancel ride
POST	<code>/rides/:id/driver-cancel</code>	Driver	Driver cancels ride
POST	<code>/rides/rate</code>	Passenger	Rate driver
GET	<code>/rides/ratings/received</code>	Driver	Get ratings received
GET	<code>/rides/history</code>	Required	Get ride history
GET	<code>/rides/nearby-drivers</code>	Public	Get nearby available drivers
GET	<code>/rides/route-preview</code>	Public	Get route polyline + ETA
POST	<code>/rides/:id/messages</code>	Required	Send chat message
GET	<code>/rides/:id/messages</code>	Required	Get chat history

Driver Endpoints

Method	Endpoint	Auth	Description
GET	/driver/dashboard	Driver	Get dashboard data
GET	/driver/status	Driver	Get online/offline status
POST	/driver/toggle-status	Driver	Toggle online/offline
POST	/driver/location-update	Driver	Update GPS location
GET	/driver/cars	Driver	Get vehicle list
POST	/driver/cars	Driver	Add vehicle
PATCH	/driver/cars/:id/set-active	Driver	Set active vehicle
GET	/driver/earnings-summary	Driver	Get earnings summary

Wallet Endpoints

Method	Endpoint	Auth	Description
GET	/wallet/accounts	Required	Get funding accounts
POST	/wallet/accounts	Required	Add account
DELETE	/wallet/accounts/:id	Required	Delete account
POST	/wallet/deposit	Required	Deposit to wallet
POST	/wallet/withdraw/request	Required	Request withdrawal
POST	/wallet/withdraw/confirm	Required	Confirm withdrawal
GET	/wallet/transactions	Required	Get transaction history

Admin Endpoints

Method	Endpoint	Auth	Description
GET	/admin/stats	Admin	Get dashboard KPIs
GET	/admin/users	Admin	Search users
PATCH	/admin/users/:id	Admin	Moderate user
DELETE	/admin/users/:userId	Admin	Cascade-delete user
GET	/admin/rides	Admin	Search rides
GET	/admin/reports	Admin	Search reports
PATCH	/admin/reports/:id	Admin	Update report status
GET	/admin/transactions	Admin	Search transactions
PATCH	/admin/transactions/:id/flag	Admin	Flag/unflag transaction
GET	/admin/audit	Admin	Get audit log

Other Endpoints

Method	Endpoint	Auth	Description
POST	/api/upload	No auth	Upload file to Cloudinary
POST	/switch-role	Required	Switch passenger/driver role
POST	/driver-application/submit	Required	Submit driver application
GET	/driver-application/me	Required	Get own application status
GET	/vehicles	Public	Get vehicle types
POST	/reports	Required	Submit report

HTTP Status Codes

Code	Meaning
200	Success
400	Bad request / validation error
401	Unauthenticated / invalid token
403	Forbidden (blocked, suspended, role mismatch)
404	Not found
409	Conflict (already registered)
429	Rate limited
500	Internal server error

7. Features Status

Phase 6 — Authentication (6/6)

Feature	Status	Notes
Email OTP login		Email step with Send Code + OTP input
Google Sign-In		Firebase SHA-1 configured, verified working
Password reset via email		3-step: email → OTP → new password
Refresh token flow		Full rotation + revocation, 7-day TTL
Session management		Auth interceptor, auto-refresh, block handling
Delete account		Self-deletion endpoint + settings UI

Phase 7 — Notifications FCM (12/14)

Event	Status	Notes
Ride accepted	✓	FCM to passenger
Driver verified	✓	FCM to driver on admin approval
Driver rejected	✓	FCM to driver on admin rejection
Ride cancelled	✓	FCM to other party
Driver arrived	✓	FCM to passenger
Passenger onboard	✓	FCM to passenger
Trip started	✓	FCM to passenger
Trip completed	✓	FCM to both parties
New chat message	✓	FCM to recipient
Wallet deposit	✓	FCM to user
Wallet withdrawal	✓	FCM to user
Payment received	✓	FCM to driver
Refund processed	✗	Not implemented
Report resolved	✗	Not implemented
Admin broadcast	✗	Not implemented

Phase 8 — Ride Experience (9/10 ✓)

Feature	Status
Driver arriving	✓
Passenger onboard	✓
Start trip	✓
End trip	✓
Cancel ride (passenger)	✓
Cancel ride (driver)	✓
Passenger rating	✓
Driver rates passenger	
Ride history (passenger)	✓
Ride history (driver)	✓
Chat	✓

Phase 9 — Maps (7/7 ✓)

Feature	Status
Pickup marker	✓
Destination marker	✓
Route preview	✓
Static driver location	✓
ETA on refresh	✓
Nearby drivers display	✓
Polling (15-30s)	✓

Phase 10 — Wallet (9/9 ✓)

Feature	Status
Top up	✓
Withdraw	✓
Earnings display	✓
Receipts	✓
Transaction history	✓
Add/delete accounts	✓
Deposit (passenger)	✓
Transfer	✓
Refund display (credit)	✓

Phase 11 — Driver Features (8/8 ✓)

Feature	Status
Vehicle management	✓
Driver documents	✓
Online/Offline toggle	✓
Driver availability	✓
Earnings dashboard	✓
Wallet (driver)	✓
Driver application flow	✓
Driver verification status	✓

Phase 12 — Admin (7/9 ✓)

Feature	Status
Dashboard statistics	✓
User search	✓
Driver search	✓
Ride search	✓
Wallet management	✓
Broadcast notifications	✖
CSV export	✖
Audit logs	✓
Report management	✓
Tools	✓
Admin web panel	✓

Phase 13 — Production (3/7 ✓)

Feature	Status
Crash reporting	✖
Analytics	✖
Backups	✖ (MongoDB Atlas auto)
Health checks	✓
Rate limiting	✓
Database indexes	✓
Monitoring	✖
Deployment	✓
ObjectId fix	✓

8. Testing

Testing Methodology

- 1. **Static Analysis:** `flutter analyze` for Dart code quality
- 2. **Manual API Testing:** curl commands against production endpoints
- 3. **Integration Testing:** Full-stack manual test flows
- 4. **Visual Testing:** Flutter widget rendering verification

API Test Cases (36 tests, 100% pass)

Area	Tests	Passed	Failed
Authentication	12	12	0
Ride Lifecycle	9	9	0
Admin Operations	6	6	0
Wallet	4	4	0
Driver Features	4	4	0
Upload	1	1	0

Flutter Analysis

```
flutter analyze
```

Result: 0 errors, 0 warnings (only info-level suggestions)

Sample curl Tests

Health Check

```
curl https://carpooling-app-3-virid.vercel.app/api/health
```

Email OTP Login

```
# Send OTP
curl -X POST https://carpooling-app-3-virid.vercel.app/api/auth/email/send-otp \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com"}'

# Verify OTP
curl -X POST https://carpooling-app-3-virid.vercel.app/api/auth/email/verify-otp \
  -H "Content-Type: application/json" \
  -d '{"email":"test@example.com","code":"123456"}'
```

Ride Lifecycle

```
# Create ride (passenger)
curl -X POST https://carpooling-app-3-virid.vercel.app/api/rides/create \
  -H "Authorization: Bearer <token>" \
  -H "Content-Type: application/json" \
  -d '{"pickup":{"lat":30.0444,"lng":31.2357,"address":"Cairo"},"destination":{"lat":30.0764,"lng":31.2357,"address":"Cairo"}}'

# Accept ride (driver)
curl -X POST https://carpooling-app-3-virid.vercel.app/api/rides/<rideId>/accept \
  -H "Authorization: Bearer <driverToken>"

# End trip
curl -X POST https://carpooling-app-3-virid.vercel.app/api/rides/end \
  -H "Authorization: Bearer <driverToken>" \
  -H "Content-Type: application/json" \
  -d '{"rideId":"<rideId>"}'
```

Admin

```
# Login
curl -X POST https://carpooling-app-3-virid.vercel.app/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"youssef@gmail.com","password":"<admin_password>"}'

# Dashboard stats
curl https://carpooling-app-3-virid.vercel.app/api/admin/stats \
  -H "Authorization: Bearer <adminToken>"

# Block user
curl -X PATCH https://carpooling-app-3-virid.vercel.app/api/admin/users/<userId> \
  -H "Authorization: Bearer <adminToken>" \
  -H "Content-Type: application/json" \
  -d '{"is_blocked":true,"block_reason":"Violation"}'

# Delete user
curl -X DELETE https://carpooling-app-3-virid.vercel.app/api/admin/users/<userId> \
  -H "Authorization: Bearer <adminToken>"
```

9. Deployment

Vercel (Production)

URL: <https://carpooling-app-3-virid.vercel.app>

Deploy

```
cd backend  
npx vercel deploy --prod
```

Environment Variables

Variable	Value	Notes
MONGODB_URI	<code>mongodb+srv:// ...</code>	MongoDB Atlas connection string
JWT_SECRET	<code>your-secret</code>	JWT signing key
JWT_EXPIRY	<code>15m</code>	Access token TTL
REFRESH_TOKEN_SECRET	<code>your-secret</code>	Refresh token signing key
GOOGLE_OAUTH_WEB_CLIENT_ID	From Firebase	Google Sign-In
GOOGLE_OAUTH_ANDROID_CLIENT_ID	From Firebase	Google Sign-In
GOOGLE_OAUTH_IOS_CLIENT_ID	From Firebase	Google Sign-In
FCM_SERVER_KEY	From Firebase	Push notifications
FCM_SENDER_ID	From Firebase	Push notifications
FCM_PROJECT_ID	From Firebase	Push notifications
CLOUDINARY_CLOUD_NAME	Cloudinary	File uploads
CLOUDINARY_API_KEY	Cloudinary	File uploads
CLOUDINARY_API_SECRET	Cloudinary	File uploads
SMTP_HOST	<code>smtp.gmail.com</code>	Email sending
SMTP_PORT	<code>587</code>	Email sending
SMTP_USER	Gmail address	Email sending
SMTP_PASS	Gmail app password	Email sending
ADMIN_PASSWORD_YOUSSEF	bcrypt hash	Admin login
ADMIN_PASSWORD_YOUSSEF1	bcrypt hash	Admin login
RATE_LIMIT_WINDOW_MS	<code>900000</code>	15 min window
RATE_LIMIT_MAX	<code>500</code>	Global max requests
AUTH_RATE_LIMIT_WINDOW_MS	<code>900000</code>	15 min window
AUTH_RATE_LIMIT_MAX	<code>30</code>	Auth max requests

vercel.json

```
{
  "builds": [{ "src": "src/index.js", "use": "@vercel/node" }],
  "routes": [
    { "src": "/admin-ui/(.*)", "dest": "/public/admin-ui/$1" },
    { "src": "/admin-ui", "dest": "/public/admin-ui/index.html" },
    { "src": "/(.*)", "dest": "/src/index.js" }
  ]
}
```

.vercelignore

```
node_modules/
apps/
*.md
.DS_Store
.git
```

Local Development

```
# Backend
cd backend && npm install && npm run dev

# Flutter
cd apps/mobile-flutter && flutter pub get && flutter run
```

Database

- **MongoDB Atlas:** Free M0 cluster
 - Auto-backups enabled
 - 86 users, 37 rides, 92 wallets (current)
 - Indexes auto-created on startup via `ensureMongoIndexes()`
 - Additional indexes via `npm run init:indexes`
-

10. Production Polish

Priority 1 — Block/Suspend Enforcement

- Added `blockCheck` middleware to `GET /auth/me`
- Flutter auth interceptor handles 403 `ACCOUNT_BLOCKED` / `ACCOUNT_SUSPENDED`
- Blocked users cannot restore sessions on app restart

Priority 2 — Driver Role Auto-Switch on Approval

- Admin approval now sets `user.role = "driver"` and `user.active_role = "driver"` automatically
- No manual role toggle needed after approval

Priority 3 — Wallet Refund Display

- `ride_refund` now displays as credit (green +) in all 3 wallet screens
- Translation keys added for EN (`"Refund"`) and AR (`"استرداد"`)

Priority 4 — Driver Application UI Differentiation

- Rejected state shows red banner vs pending amber banner
- `DriverVerificationStatusScreen` shows rejection card with admin review note
- All 4 `driverApplicationStatus` values handled: `none` , `pending` , `approved` , `rejected`

Delete Account Feature

- `POST /auth/delete-account` with password verification + full cascade delete
 - Settings UI with password dialog (email users) or confirmation dialog (Google/phone users)
 - All associated data cleaned up (same as admin delete)
-

11. Changelog

v1.0.0 — Production Polish Release

Authentication

- Email OTP login system — replaces phone SMS as primary auth method
- Google Sign-In fully configured and verified working
- Password reset via email (3-step: email → OTP → new password)
- Refresh token rotation with anti-reuse detection
- Self-delete account endpoint + settings UI

Security

- Block/suspend enforcement on `GET /auth/me`
- Flutter auth interceptor handles 403 ACCOUNT_BLOCKED/SUSPENDED
- Rate limiting: 500/15m global, 30/15m on auth routes
- JWT 15min expiry with 7-day refresh tokens

Ride Experience

- Complete ride lifecycle: requesting → accepting → arriving → onboard → start → end
- Passenger rating of driver on trip completion
- In-app chat with 10s polling
- All transitions send FCM push notifications

Wallet

- Top up, withdraw (2-step with OTP), transaction history
- Ride refund displayed as credit (green +)
- Wallet accounts: cash, instapay, vodafone, card

Driver Features

- Vehicle management (list, add, set active)

- Online/offline toggle, location tracking, earnings dashboard
- Application flow: submit → pending → approved/rejected
- Rejected state UI with admin review note

Admin

- Dashboard with KPIs, charts, activity feed
- User/ride/report/transaction/audit search with pagination
- User moderation with driver approval auto role-switch
- Transaction flagging

Infrastructure

- Custom ODM over MongoDB native driver — ObjectId auto-conversion
- MongoDB Atlas with 30+ auto-created indexes
- Health check endpoint at `/api/health`
- Vercel production deployment with `.vercelignore`

Bug Fixes

- ODM ObjectId/string `_id` mismatch — root cause of 401 USER_NOT_FOUND
 - Flutter login screen Form wrappers — `_formKey.currentState` was null
 - Driver onboarding validation feedback — added snackbar
 - Admin Google sign-in token key mismatch
-

12. Conclusion & Future Work

Conclusion

The Carpooling Application has been successfully developed and deployed to production at <https://carpooling-app-3-virid.vercel.app>. The project delivers a full-stack ride-hailing experience with:

Achievements:

- Complete auth system (Email OTP, Google Sign-In, phone OTP) with secure JWT + refresh tokens
- Full ride lifecycle with FCM push notifications at every state
- Wallet system with top-up, withdrawal, and transaction history
- Driver management with vehicle CRUD, earnings dashboard, and application flow
- Admin panel with dashboard KPIs and full user/ride/report management
- 87% feature completion (61/70 planned features)
- Bilingual support (English + Arabic) with RTL compatibility
- 0 errors, 0 warnings in Flutter static analysis

Key Metrics:

- 50+ API endpoints across 10 route modules
- 30+ screens, 20+ reusable widgets
- 15 database collections with 30+ indexes
- 36 API tests with 100% pass rate

Future Work

Priority	Feature	Effort
High	Driver rates passenger (endpoint + UI)	Medium
High	Admin broadcast notifications	Medium
Medium	Refund processed FCM notification	Small
Medium	Report resolved FCM notification	Small
Medium	CSV export for admin	Medium
Medium	Crash reporting (Sentry / Firebase Crashlytics)	Small
Medium	Analytics integration	Medium
Low	Foreground lifecycle refresh on FCM	Small
Low	Admin web UI JWT refresh	Small
Low	My Reports screen for users	Medium
Low	Monitoring (uptime, error alerts)	Small
Blocked	Firebase billing for phone SMS	Small

References

1. Flutter Documentation — <https://docs.flutter.dev>
2. Riverpod State Management — <https://riverpod.dev>
3. Dio HTTP Client — <https://pub.dev/packages/dio>
4. GoRouter — https://pub.dev/packages/go_router
5. Flutter Map (OpenStreetMap) — <https://docs.fleaflet.dev>
6. Firebase Cloud Messaging — <https://firebase.google.com/docs/cloud-messaging>
7. MongoDB Atlas — <https://www.mongodb.com/atlas>
8. Express.js — <https://expressjs.com>
9. Vercel Serverless — <https://vercel.com/docs>
10. Nodemailer — <https://nodemailer.com>
11. Cloudinary — <https://cloudinary.com>
12. Google Sign-In for Flutter — https://pub.dev/packages/google_sign_in
13. Easy Localization — https://pub.dev/packages/easy_localization
14. JWT (jsonwebtoken) — <https://github.com/auth0/node-jsonwebtoken>
15. bcryptjs — <https://github.com/dcodeIO/bcrypt.js>

End of Complete System Documentation